



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 3 · SESIÓN 1

Tipos de Datos

Bienvenidos al Módulo II: programar de verdad

Módulo I (Semanas 1–2): *por qué* programar y *cómo* organizar un proyecto. **Desde hoy:** escribimos R en serio.

La semana pasada **nombramos** los tipos de datos. Hoy los **dominamos** — son la base de todo lo que viene.

Al terminar esta sesión podrás:

- ▶ Explicar por qué el **tipo** de una variable decide qué puedes hacer con ella
- ▶ Crear y reconocer los cinco tipos fundamentales en R
- ▶ **Convertir** entre tipos y anticipar cuándo aparece un `NA`
- ▶ **Verificar** el tipo de cualquier objeto con `class()`, `typeof()` e `is.*()`

💬 **Activación:** la mayoría de los errores “raros” al cargar datos reales son, en el fondo, problemas de tipo. Hoy aprendemos a verlos venir.

Bloque A — ¿Qué es un tipo de dato?

El tipo decide qué se puede hacer

Cada valor en R tiene un **tipo**, y el tipo determina qué operaciones son válidas. Es la regla más importante para entender los errores.

No se puede “sumar” una región a un salario, igual que no se suman peras con kilómetros. R lo impide por diseño:

```
2000 + 500           # numeric: aritmética válida
```

```
[1] 2500
```

```
"Ñuble" + 1          # character: no tiene sentido sumar texto
```

```
Error in `"Ñuble" + 1`:  
! non-numeric argument to binary operator
```

```
TRUE + TRUE          # logical: TRUE vale 1 → sirve para CONTAR
```

```
[1] 2
```

💬 **Pregunta:** ¿por qué `TRUE + TRUE` da 2? ¿Y por qué eso es útil para contar cuántos hogares son pobres? (lo vemos en el Bloque B)

El costo de equivocarse de tipo

El 90% de los errores “raros” al cargar datos reales (CASEN, Banco Central) son, en el fondo, **problemas de tipo**:

Ingreso leído como texto → no se puede promediar. `mean()` falla o devuelve `NA`.

Año leído como `factor` → se grafica mal en el eje del tiempo.

Un `NA` escondido como `"."` o `"s/i"` → contamina silenciosamente los cálculos.

Dominar los tipos no es teoría: es lo que te ahorra **horas de depuración** con datos reales.

Bloque B — Los cinco tipos fundamentales

numeric : la columna vertebral del análisis

Guarda números, con o sin decimales. En R casi todo número es `double`. Admite toda la aritmética y las funciones matemáticas.

```
salario <- 850000  
ipc <- 1.043  
salario / ipc           # salario real
```

```
[1] 814956.9
```

```
log(salario)           # log-ingreso: uso típico en econometría
```

```
[1] 13.65299
```

Existe también `integer` (se escribe `2L`), pero en la práctica casi siempre se trabaja con `numeric`. No hay que preocuparse por la distinción todavía.

 IMAGEN: regla/termómetro numérico, o un gráfico de una serie económica.

character : texto y etiquetas

Guarda texto, **siempre entre comillas** (`" ... "`). No se opera aritméticamente: se manipula.

```
region <- "Ñuble"  
sector <- "Agricultura"  
paste(sector, "-", region)      # pegar textos
```

```
[1] "Agricultura - Ñuble"
```

```
nchar(region)                  # cuántos caracteres
```

```
[1] 5
```

Trampa clásica: un número que viene como texto (`"480609"`) no se puede sumar. Hay que **convertirlo** antes — lo vemos en el Bloque C.

logical : verdadero/falso y el poder de contar

Solo dos valores: `TRUE` / `FALSE` . Nacen de **comparaciones** (`>` , `<` , `==` , `!=`).

```
ingreso <- c(120000, 480000, 350000, 90000)
linea_pobreza <- 216000
ingreso < linea_pobreza      # TRUE = bajo la línea de pobreza
```

```
[1] TRUE FALSE FALSE TRUE
```

El truco clave: `TRUE` vale 1 y `FALSE` vale 0.

```
sum(ingreso < linea_pobreza)  # cuántos bajo la línea
```

```
[1] 2
```

```
mean(ingreso < linea_pobreza) # proporción → tasa de pobreza
```

```
[1] 0.5
```

Este patrón — `sum()` o `mean()` de una condición — es uno de los más usados en **todo** el análisis económico aplicado.

factor : variables categóricas con orden

Para categorías con un conjunto **finito y conocido** de niveles (`levels`). A diferencia de `character`, el `factor` recuerda todas las categorías posibles y puede tener **orden**.

```
educ <- factor(c("media", "superior", "básica", "media"),
              levels = c("básica", "media", "superior"),
              ordered = TRUE)
levels(educ)      # las categorías, en su orden
```

```
[1] "básica"  "media"   "superior"
```

```
educ < "superior"  # el orden importa: comparar niveles educativos
```

```
[1] TRUE FALSE TRUE TRUE
```

Clave para econometría: las regresiones tratan los `factor` como variables *dummy* automáticamente. Volveremos a esto en el Módulo II.

Date : el tiempo como dato

Las fechas **no son texto**: R las guarda como días desde el 1970-01-01, lo que permite **restar, ordenar y graficar** en el tiempo. Se crean con `as.Date()` en formato ISO `"AAAA-MM-DD"`.

```
fecha <- as.Date("2026-03-31")  
class(fecha)
```

```
[1] "Date"
```

```
fecha - as.Date("2026-01-01")      # diferencia en días
```

```
Time difference of 89 days
```

```
format(fecha, "%B %Y")              # mes y año con formato
```

```
[1] "marzo 2026"
```

Si las fechas se cargan como **texto**, se ordenan alfabéticamente, no cronológicamente → desastre en las series de tiempo. Conviértelas siempre.

Bloque C — Conversión entre tipos

Las funciones `as.*()`

Convertir (o “castear”) es transformar un valor de un tipo a otro de forma **explícita y controlada**.

Función	Convierte a
<code>as.numeric()</code>	número
<code>as.character()</code>	texto
<code>as.logical()</code>	TRUE / FALSE
<code>as.factor()</code>	categoría
<code>as.Date()</code>	fecha

```
poblacion <- "480609"      # texto
as.numeric(poblacion) + 1  # ahora sí: ya es número
```

```
[1] 480610
```

```
as.character(2026)        # número → texto
```

```
[1] "2026"
```

Cuando la conversión produce NA

Convertir a número un texto que **no** es numérico devuelve `NA` (con un *warning*), no un error. Hay que anticiparlo y revisarlo.

```
ingresos_crudos <- c("120000", "350000", "s/i", "90.000")  
as.numeric(ingresos_crudos)
```

```
Warning: NAs introduced by coercion
```

```
[1] 120000 350000      NA      90
```

- ▶ `"s/i"` → `NA` (esperado: no es número)
- ▶ `"90.000"` → `NA` (el punto se interpreta como separador raro)

Caso real de encuestas: separadores de miles (`.`), símbolos (`$`), "no responde" (`s/i` , `99`). **Limpiar el texto antes de convertir.**

En la **Semana 6** trabajamos los `NA` a fondo. Hoy basta con reconocer cuándo aparecen y por qué.

Coerción automática: lo que R hace sin avisar

R también convierte **solo**, siguiendo la jerarquía `logical → numeric → character`.
Conviene conocerla.

```
c(1, 2, "tres")           # todo se vuelve character
```

```
[1] "1"    "2"    "tres"
```

```
c(TRUE, 5, 10)           # TRUE se vuelve numeric (1)
```

```
[1] 1  5 10
```

Regla: en un vector todos los elementos comparten **un único tipo**. Si mezclas, R “promociona” al más general (normalmente `character`).

Por eso una sola celda de texto en una columna numérica vuelve **toda** la columna texto. Ese es el `summary()` que salía “mal” en el Bloque A.

Bloque D — Verificación

Preguntarle a R qué tipo es

Antes de operar, **verifica**. Tres herramientas según la necesidad:

```
x <- c(120000, 480000, 350000)
class(x)          # tipo de alto nivel: lo que normalmente importa
```

```
[1] "numeric"
```

```
typeof(x)         # representación interna
```

```
[1] "double"
```

```
is.numeric(x)     # pregunta sí/no, ideal para chequeos
```

```
[1] TRUE
```

```
str(x)            # resumen compacto: tipo + dimensión + primeros valores
```

```
num [1:3] 120000 480000 350000
```

Hábito profesional: tras cargar cualquier dato real, corre `str()` para auditar los tipos columna por columna **antes** de analizar.

Cierre: lo que nos llevamos hoy

Tipo	Para qué	Verificar con
<code>numeric</code>	cálculos, montos, tasas	<code>is.numeric()</code>
<code>character</code>	texto, glosas, códigos	<code>is.character()</code>
<code>logical</code>	condiciones, contar / proporciones	<code>is.logical()</code>
<code>factor</code>	categorías (con orden)	<code>is.factor()</code>
<code>Date</code>	tiempo, series, antigüedad	<code>inherits(x, "Date")</code>

En una línea: **el tipo decide qué puedes hacer; verifica antes de calcular.**

Puente a la Sesión 2: hoy vimos valores sueltos. Ahora los vamos a juntar en **vectores** y a seleccionar subconjuntos con índices y condiciones.



Sesión 1 — Fin

A continuación: **Sesión 2 — Vectores y Subsetting** `c()` ·
indexación · operaciones vectorizadas